hashlock.

Security Audit

DB Cherry (AA Wallet)

May 2025 info@hashlock.com.au Hashlock Pty Ltd

Table of Contents

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

Executive Summary

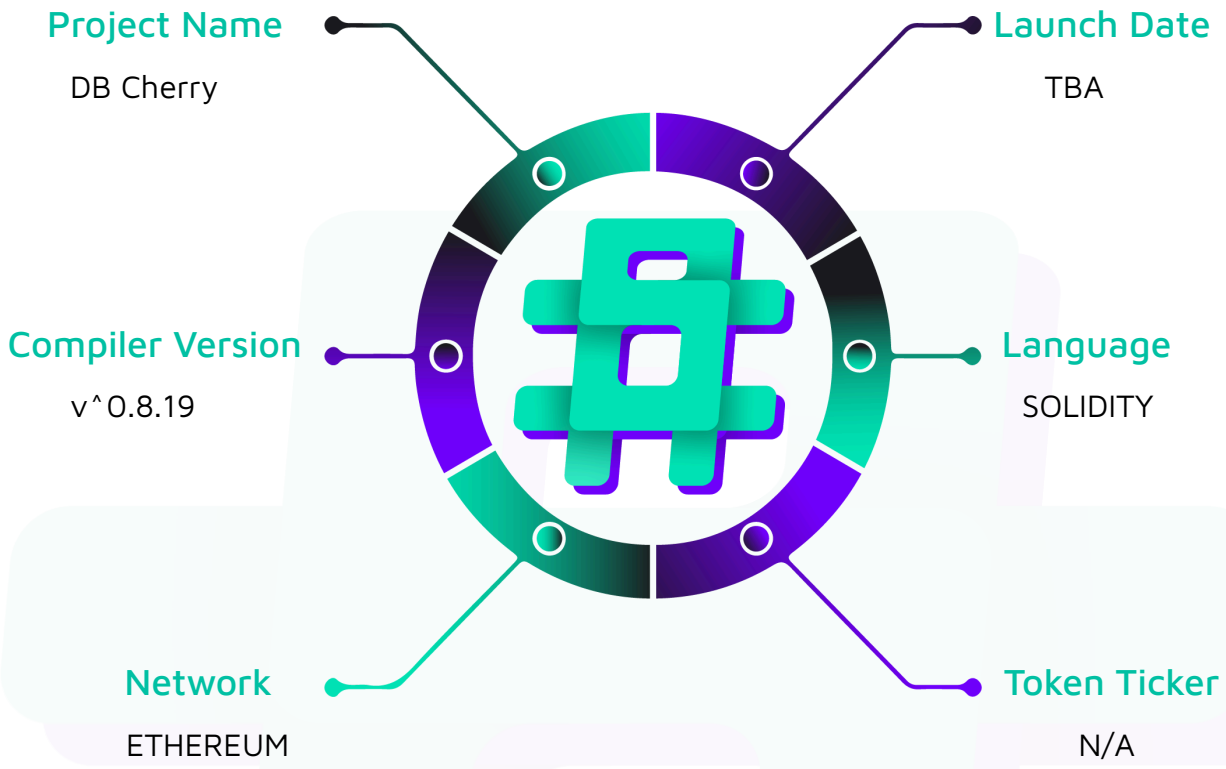
The DB Cherry team partnered with Hashlock to conduct a security audit of their smart contracts. Hashlock manually and proactively reviewed the code in order to ensure the project's team and community that the deployed contracts are secure.

Project Context

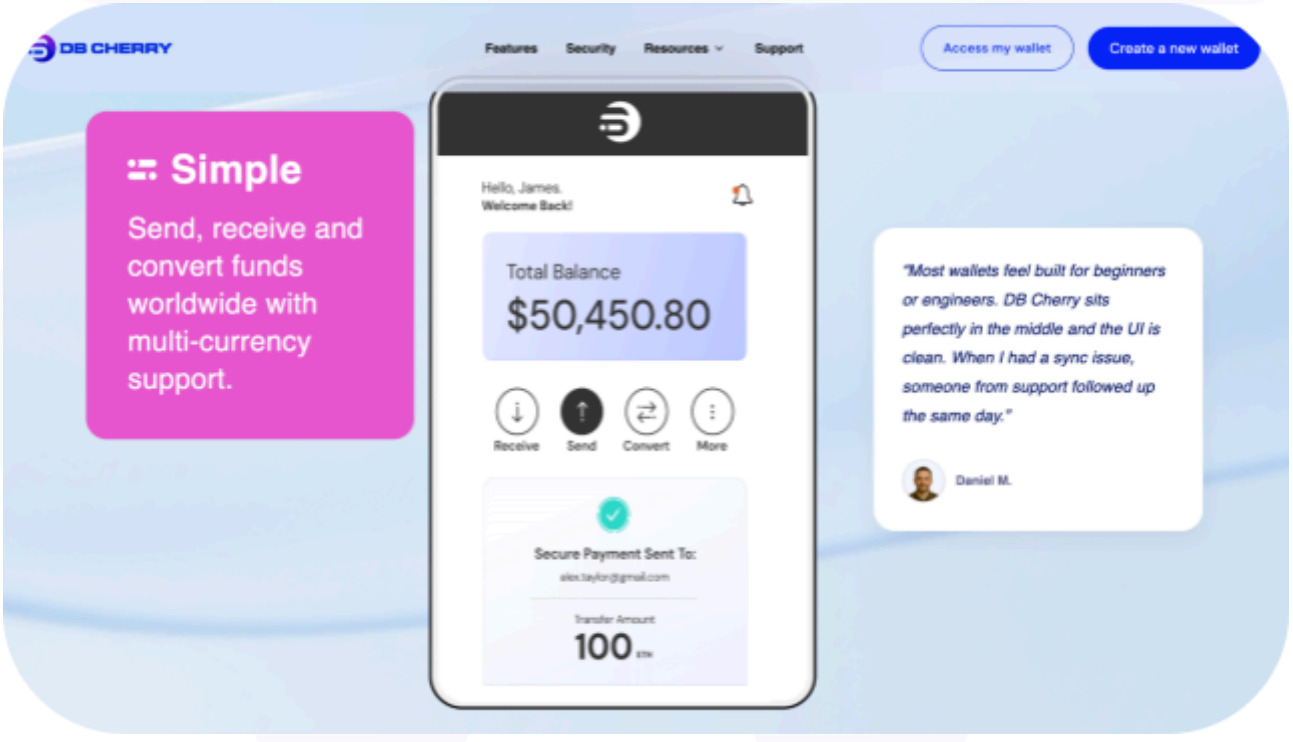
DB Cherry is a next-generation digital wallet designed to seamlessly bridge the gap between cryptocurrencies and traditional fiat currencies. It offers users the ability to send, receive, and convert funds globally, supporting multiple currencies including ETH, MATIC, USDT, and USDC across various blockchain standards. The platform emphasizes user-friendly features such as automatic savings through purchase round-ups, transparent low fees, and competitive exchange rates. Security is a cornerstone of DB Cherry, incorporating advanced measures like instant backup and auto-wipe capabilities in case of compromise, as well as decentralized identity integration for secure authentication. With 24/7 expert support, DB Cherry aims to provide a reliable and efficient financial tool for both novice and experienced users navigating the digital finance landscape.

Project Name: DB Cherry
Project Type: AA Wallet.
Compiler Version: ^0.8.19
Website: https://sandbox.dbcherry.com/
Logo:

Visualised Context:



Project Visuals:



Audit Scope

We at Hashlock audited the solidity code within the DB Cherry project, the scope of work included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

Description	DB Cherry Smart Contracts
Platform	Ethereum / Solidity
Audit Date	May, 2025
Contract 1	SmartWallet.sol
Contract 1 MD5 Hash	725a5c4e19d831654ea90ec8a549f81b
Contract 2	SmartWalletFactory.sol
Contract 2 MD5 Hash	946f0fbf4a8fca5466dc7ef04c301987
Contract 3	SmartWalletProxy.sol
Contract 3 MD5 Hash	5d7aeb46efa0c40f9e15ccc58cd6fdea
Audited GitHub Commit Hash	0e8f1033776bb389632c3264204b10b6595b5147
Fix Review GitHub Commit Hash	37c131e0d5cc8e405367ed96415a31d0a93ecb0e

Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Secure"**. The contracts all follow simple logic, with correct and detailed ordering. They use a series of interfaces, and the protocol uses a list of Open Zeppelin contracts.

The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the Audit Findings section. The list of audited assets is presented in the Audit Scope section and the project's contract functionality is presented in the Intended Smart Contract Functions section.

All vulnerabilities initially identified have now been resolved.

Hashlock found:

1 High severity vulnerabilities

1 Medium severity vulnerabilities

3 Low severity vulnerabilities

1 QA

Caution: *Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.*

Intended Smart Contract Functions

Claimed Behaviour	Actual Behaviour
SmartWallet.sol - Creates upgradeable smart contract wallets - Validates operations via signature verification - Executes single and batched transactions - Manages gas payments and nonces - Owner-controlled upgrades	**Contract achieves this functionality.**
SmartWalletFactory.sol - Creates new smart wallet instances - Personalizes wallets with owner and entry point parameters - Computes wallet addresses before deployment - Emits events when new wallets are created	**Contract achieves this functionality.**
SmartWalletFactory.sol - Implements proxy pattern for smart wallet deployment	**Contract achieves this functionality.**

Code Quality

This audit scope involves the smart contracts of the DB Cherry project, as outlined in the Audit Scope section. All contracts, libraries, and interfaces mostly follow standard best practices and to help avoid unnecessary complexity that increases the likelihood of exploitation, however, some refactoring was recommended to optimize security measures.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the DB Cherry project smart contract code in the form of Github access.

As mentioned above, code parts are well commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments are helpful in providing an understanding of the protocol's overall architecture.

Dependencies

As per our observation, the libraries used in this smart contracts infrastructure are based on well-known industry standard open source projects.
Apart from libraries, its functions are used in external smart contract calls.

Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies.
QA	Quality Assurance (QA) findings are informational and don't impact functionality. Supports clients improve the clarity, maintainability, or overall structure of the code.

Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

Significance	Description
Resolved	The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue.
Acknowledged	The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception.
Unresolved	The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.

Audit Findings

High

[H-01] SmartWallet#validateUserOp - Missing Authorization in validateUserOp Allows Front-running and Fund Theft

Description

The SmartWallet contract's validateUserOp() function lacks authorization check to verify the caller is the trusted EntryPoint. This allows attackers to front-run legitimate transactions and drain funds from the contract..

Vulnerability Details

The validateUserOp() function Validates user operations by verifying owner's signature, checking nonce, and funding the entry point if needed.

```
function validateUserOp(
    PackedUserOperation calldata userOp,
    bytes32 userOpHash,
    uint256 missingAccountFunds
  ) external override returns (uint256) {
    require(
      userOpHash.toEthSignedMessageHash().recover(userOp.signature) ==
        owner(),
      "Invalid signature"
    );
    if (userOp.nonce != nonce) revert("Invalid nonce");
    nonce++;
    if (missingAccountFunds > 0) {
      (bool sent, ) = payable(msg.sender).call{
```

```
            value: missingAccountFunds
        }("");
        require(sent, "Funding failed");
    }
    return 0;
}
```

This function has no verification that `msg.sender` is the `trustedEntryPoint`. While the function checks if the signature is valid, an attacker can simply front-run a legitimate transaction by copying its exact parameters (including the valid signature) and submitting it with a higher gas price. The attacker can set missingAccountFunds to the contract's entire balance, and since there's no check on who calls the function, the contract will send these funds to the caller.

Impact

This vulnerability allows an attacker to drain the entire contract balance through a simple front-running attack due to missing caller validation checks.

Recommendation

Add this authorization check at the beginning of the validateUserOp function:

```
require(msg.sender == trustedEntryPoint, "Not entry point");
```

Status

Resolved

Medium

[M-01] SmartWalletFactory#createWallet - Wallet Creation is Vulnerable to Front-Running DoS Attacks

Description

The createWallet function in SmartWalletFactory.sol is vulnerable to front-running attacks. An attacker can observe pending wallet creation transactions in the mempool and execute them first with the same parameters, causing the legitimate user's transaction to fail.

Vulnerability Details

The createWallet() function creates wallets with deterministic addresses using CREATE2 and a salt derived from owner address and user-provided salt.

```
function createWallet(
    address owner,
    address entryPoint,
    uint256 salt
  ) external returns (address wallet) {
    bytes memory initData = abi.encodeWithSignature(
      "initialize(address,address)",
      entryPoint,
      owner
    );
    bytes32 finalSalt = keccak256(abi.encodePacked(owner, salt));
    wallet = address(
      new SmartWalletProxy{salt: finalSalt}(implementation, initData)
    );
    emit WalletCreated(wallet, owner);
```

```
}
```

Since wallet addresses are determined solely by these inputs, an attacker who observes a pending transaction can submit their own transaction with identical parameters and a higher gas price to ensure it's mined first. This creates the wallet at the intended address, causing the legitimate transaction to revert since the contract already exists at that address.

Impact

Denial of service preventing users from creating wallets, resulting in wasted gas fees and poor user experience..

Recommendation

include sender-specific information in the salt calculation to prevent others from using the same salt, such as:

```
bytes32 finalSalt = keccak256(abi.encodePacked(owner, salt, msg.sender));
```

Status

Resolved

Low

[L-01] Contracts - Use of floating pragma

Description

The contracts SmartWallet.sol,SmartWalletFactory.sol and SmartWalletProxy.sol use a floating pragma version (^0.8.19). It's crucial to compile and deploy contracts with the same compiler version and settings used during development and testing. Fixing the pragma version prevents compilation with different versions. Using an outdated pragma version could introduce bugs that negatively affect the contract system, while newly released versions may have undiscovered security vulnerabilities.

Recommendation

Change the pragma statement in contracts to a fixed version, such as pragma solidity 0.8.19;. This locks the contract to a specific compiler version.

Status

Resolved

[L-02] Contracts - Missing Initializer Call for Inherited UUPSUpgradeable Contract

Description

The SmartWallet contract inherits from OpenZeppelin UUPSUpgradeable contract but does not call the recommended initializer function during its initialization process.

Recommendation

call `__UUPSUpgradeable_init()` in initialize function..

Status

Resolved

[L-03] Contracts - Missing Input Validation in SmartWallet initialize() function

Description

The SmartWalllet contract's initialize() function does not validate any of its input parameters (_entryPoint , _owner). This lack of validation could allow the contract to be deployed in an inconsistent or invalid state. For example, the contract could be initialized with a zero addresses.

```
function initialize(
        address _entryPoint,
        address _owner
    ) public initializer {
        trustedEntryPoint = _entryPoint;
        __Ownable_init();
        _transferOwnership(_owner);
    }
```

Recommendation

Add input validation checks in the initialize function.

Status

Resolved

QA

[Q-01] Sio2AdapterAssetManager - Missing Event Emissions in SmartWallet Contract

Description

The SmartWallet contract does not emit events for critical state-changing operations such as operation validation, nonce increments, and fund transfers. This reduces transparency, makes off-chain monitoring impossible, complicates debugging, and limits integration with external tools and services. Event emissions are considered best practice for smart contracts.

Recommendation

Add appropriate event to the contract and emit them in the validateUserOp function

Status

Resolved

Centralisation

The DB Cherry project values security and utility over decentralisation.

The owner executable functions within the protocol increase security and functionality but depend highly on internal team responsibility.

Conclusion

After Hashlock's analysis, the DB Cherry project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au
Contact: info@hashlock.com.au